

บทที่ 3

เว็บเซอร์วิส

เว็บเซอร์วิส (Web Services) คือ แอปพลิเคชัน หรือ โปรแกรมที่ทำงานอย่างใดอย่างหนึ่ง ในลักษณะให้บริการ โดยจะถูกเรียกใช้งานจาก แอปพลิเคชัน อื่นๆ ในรูปแบบ RPC (Remote Procedure Call) ซึ่งการเปลี่ยนคือ XML ทำให้เราสามารถเรียกใช้คอมพิวเตอร์ใดๆ ก็ได้ ในแพลตฟอร์ม (platform) ใดๆ ก็ได้ บนโพรโทคอล HTTP ซึ่งเป็นโพรโทคอลสำหรับเวิร์ด ไซด์ เว็บ (World Wide Web) อันเป็นช่องทางที่ได้รับการยอมรับทั่วโลกในการติดต่อสื่อสารกันระหว่างแอปพลิเคชันกับแอปพลิเคชันในปัจจุบัน

เทคโนโลยีในการกระจายข่าวสารข้อมูลทางอินเทอร์เน็ต ในปัจจุบันก็คือ เว็บเพจ แต่จากการที่มันมีความสามารถที่จะทำงานได้ด้วยการรวมภาษาทั้งไคลเอนต์ (Client) และเซิร์ฟเวอร์ สไซด์ สคริปต์ (Server Side Script) ไว้ในตัวเองเช่นภาษา VBScript, Java Script หรือ ASP, PHP, JSP นั้นทำให้เว็บเพจมีลักษณะคล้ายแอปพลิเคชัน จึงถูกเรียกรวมกันว่า เว็บแอปพลิเคชัน

เว็บแอปพลิเคชัน สามารถตอบสนองความคิดทฤษฎีการประมวลผลแบบกระจาย (Distributed Processing) ได้ในระดับหนึ่งซึ่งก็คือ การแบ่งการประมวลผลไว้ที่ฝั่งไคลเอนต์ และฝั่งเซิร์ฟเวอร์ และมักจะมีการใช้ฐานข้อมูล ควบคู่กับการทำ เว็บแอปพลิเคชัน ไปด้วยตามความต้องการในการทำอี-บิซซิเนส (E-Business) และอี-คอมเมิร์ซ (E-Commerce) ที่กำลังเป็นที่นิยมในปัจจุบัน และเกิดปัญหาที่ตามมาคือ เรื่องของการจ่ายเงินหรือที่เรียกว่า อี-เพย์เมนต์ (E-payment) หรือ เพย์เมนต์-เกตเวย์ (Payment-Gateway) ซึ่งเว็บแอปพลิเคชัน ที่ทำอี-คอมเมิร์ซ ต้องใช้บริการจากธนาคารออนไลน์ ในการจัดเก็บเงินกับลูกค้า เพราะด้วยเทคโนโลยีนี้การให้บริการเก็บเงินจากธนาคารออนไลน์ จำเป็นที่ผู้ค้าต้องไปทำการตกลงกับธนาคารและเขียนโปรแกรมให้ตรงตามมาตรฐานที่ธนาคารออนไลน์ กำหนดไว้

ด้วยปัญหายุ่งยากในการค้นหา ติดต่อและตกลงในการขอใช้บริการเก็บเงินจากธนาคาร ออนไลน์ แนวคิดเว็บเซอร์วิสจึงดูเหมือนเป็นทางออกของปัญหานี้ ความเด่นของเทคโนโลยีเว็บเซอร์วิส นี้ก็คือ การทำให้เว็บกับเว็บ สามารถติดต่อสื่อสารกันได้ด้วยเอกสาร XML ที่ทั้งคนและคอมพิวเตอร์เข้าใจ และคอมพิวเตอร์ยังสามารถนำข้อมูลนั้นไปประมวลผลต่อได้ ด้วยเอกสาร XML นี้เองทำให้เว็บสามารถส่งข้อมูลที่จำเป็นไปให้อีก เว็บ หนึ่งทำงานบางอย่างให้หรือให้บริการนั่นเอง ทำให้เป็นการง่ายที่จะเขียนโปรแกรมที่จะติดต่อสื่อสารหรือขอใช้บริการเก็บเงินจากธนาคาร ออนไลน์ แต่สำหรับ เว็บแอปพลิเคชัน ที่ใช้การส่งข้อมูลเป็น html ทำให้ข้อมูลนั้นไม่สามารถนำไปใช้ได้ การเขียนโปรแกรมจึงยุ่งยากตามที่กล่าวด้านบน

แนวคิดของ เว็บเซอร์วิสก็คือ เว็บ ที่สามารถทำงานอะไรบางอย่างหรือก็คือให้บริการบางอย่างจากการร้องขอจากต่างเซิร์ฟเวอร์ ด้วยเหตุนี้ทำให้เทคโนโลยี เว็บเซอร์วิสเอื้อต่อแนวคิดทฤษฎีการประมวลผลแบบกระจาย มากกว่า เว็บแอปพลิเคชัน และเมื่อประกอบกับการที่ เว็บเซอร์วิสมี UDDI ทำให้ เว็บเซอร์วิสสามารถค้นหาบริการต่างๆ ที่ต้องการได้จากทั่วทุกมุมโลก

ในอนาคตอาจเป็นไปได้ว่า แอปพลิเคชัน ของก็อาจเป็นเพียงแค่การรวมบริการ (Service) ที่แต่ละเว็บเซอร์วิสมีบริการมาให้ใช้เท่านั้นเอง

องค์ประกอบในการใช้งาน .NET เว็บเซอร์วิส

3.1 XML

3.1.1 แนะนำ XML

XML ย่อมาจาก eXtensible Markup Language XML ถูกออกแบบมาเพื่อใช้อธิบายข้อมูล ใน XML จะไม่มีแท็ก (tag) ที่กำหนดไว้ล่วงหน้า เราจะต้องสร้างแท็กของเราขึ้นมาเอง และจะใช้ Document Type Definition (DTD) ในการอธิบายรูปแบบของข้อมูล

ข้อแตกต่างที่สำคัญระหว่าง XML และ HTML คือ XML ถูกออกแบบมาเพื่อใช้แลกเปลี่ยนข้อมูล ไม่ใช่มาแทนที่ HTML XML กับ HTML ถูกออกแบบมาเพื่อจุดประสงค์ที่ต่างกัน HTML จะเกี่ยวข้องกับการแสดงข้อมูล XML จะเกี่ยวข้องกับการอธิบายข้อมูลแท็กของ HTML เช่น <H1></H1> เป็นแท็กสำหรับบอกรูปแบบการแสดงผล ในขณะที่แท็กของ XML ต้องสร้างขึ้นมาเองเช่น

```
<BOOK>
  <NAME>Beginning XML</NAME>
  <ISDN>1-861003-41-2</ISDN>
  <PAGE>800</PAGE>
</BOOK>
```

จะใช้แสดงข้อมูลของหนังสือที่ประกอบด้วย element BOOK , NAME , ISDN และ PAGE ข้อมูลที่อยู่ในรูปแท็กนี้ เป็นอิสระต่อซอฟต์แวร์และฮาร์ดแวร์ จึงเหมาะที่จะนำมาใช้ในการแลกเปลี่ยนข้อมูลระหว่างระบบบนอินเทอร์เน็ต การเปลี่ยนข้อมูลให้อยู่ในรูป XML เป็นการลดความซับซ้อนและสามารถสร้างข้อมูลที่ถูกอ่านโดยแอปพลิเคชันใด ๆ ก็ได้ นอกจากนั้นการเพิ่มเติมอีริเมนต์ (element) จะไม่ทำให้แอปพลิเคชันเสียหาย เนื่องจากการอ่านข้อมูลโดย XML parser จะใช้วิธีแยกข้อมูลจาก element ที่ต้องการ จาก XML ข้างต้นนี้จะแยกข้อมูล element NAME , ISDN และ PAGE หากเพิ่มอีริเมนต์ AUTHOR เข้าไป แอปพลิเคชันก็ยังคงดึงข้อมูลในอีริเมนต์เดิมโดยจะมองข้ามอีริเมนต์ที่ไม่ได้กำหนดให้แยกข้อมูลออกมาในตอนพัฒนาแอปพลิเคชัน

3.1.2 ไวยากรณ์ของ XML

กฎไวยากรณ์ของ XML นั้นง่าย และเคร่งครัดมาก กฎของมันง่ายต่อการเรียนรู้ และง่ายต่อการใช้ ด้วยเหตุนี้การสร้างซอฟต์แวร์ในการอ่าน และจัดการกับข้อมูลจึงเป็นเรื่องง่าย ตัวอย่างของเอกสาร XML ดังนี้

```
<?XML version="1.0"?>

<note>

<to>Tove</to>

<from>Jani</from>

<heading>Reminder</heading>

<body>Don't forget me this weekend!</body>

</note>
```

ในบรรทัดแรกคือการประกาศเอกสาร XML ที่บอกถึง XML เวอร์ชัน

บรรทัดต่อมาคือ root element (note) อีก 4 บรรทัดถัดมาเป็นการอธิบายถึง 4 child element ของ root (to , from , heading , body) และบรรทัดสุดท้ายเป็นการกำหนดจุดสิ้นสุดของ root

กฎของเอกสาร XML มีดังนี้

- **XML Element ทุกตัวต้องมีการปิดแท็ก**

ใน HTML บาง element ไม่จำเป็นต้องมีการปิดแท็ก แต่ใน XML ทุก element จำเป็นที่จะต้องมีการปิดแท็ก

<p>This is a paragraph	X
This is a paragraph	✓

- **XML แท็กมีความแตกต่างระหว่างตัวพิมพ์เล็กและตัวพิมพ์ใหญ่ (case sensitive)**

ใน XML แท็ก <Letter> นั้นต่างจาก <letter> เนื่องจาก XML นั้นมีคุณสมบัติที่แยกความแตกต่างระหว่างตัวพิมพ์เล็กและตัวพิมพ์ใหญ่

<Message>This is incorrect</message>	X
<message>This is correct</message>	✓

- **XML Element ทุกตัวต้องมีการซ้อนกันอย่างเหมาะสม**

ใน HTML บาง element สามารถเหลื่อมกันได้ แต่ใน XML นั้นต้องซ้อนกันเป็นชั้นโดยสมบูรณ์

```
<b><i>This text is bold and italic</b></i>
```

X

```
<b><i>This text is bold and italic</i></b>
```

✓

- **ทุกเอกสาร XML ต้องมี root แท็ก**

แท็กแรกในเอกสาร XML นั้นจะเป็น root แท็กและจะมีได้เพียงหนึ่งเดียวเท่านั้น element อื่น ๆ ทั้งหมดจะต้องถูกซ้อนอยู่ใน root element เท่านั้น และ element เหล่านั้นก็สามารถมี element ย่อย (child element) ได้อีก ดังนี้

```
<root>
```

```
  <child>
```

```
    <subchild>.....</subchild>
```

```
  </child>
```

```
</root>
```

- **ค่าของ Attribute ต้องมีอยู่ภายในเครื่องหมายคำพูด (“”)**

XML element สามารถมี attribute ได้ โดยที่ค่าของ attribute จะต้องอยู่ภายในเครื่องหมายคำพูด (“”) เอกสาร XML ข้างล่างนี้ อันแรกผิด อันที่สองถูก

```
<note date=12/11/99>
```

X

```
<note date="12/11/99">
```

✓

3.1.3 XML Element

XML element มีคุณสมบัติดังนี้

- **XML Element สามารถขยายได้**

ตามตัวอย่างนี้

```
<note>
```

```
  <to>Tove</to>
```

```
  <from>Jani</from>
```

```
  <body>Don't forget me this weekend!</body>
```

```
</note>
```

ถ้าเราจะสร้างแอปพลิเคชันที่สามารถแยก element to , from และ body จาก เอกสาร XML เพื่อแสดงผลลัพธ์ดังนี้

MESSAGE

To:Tove

From: Jani

Don't forget me this weekend!

และ ถ้าเพิ่มข้อมูลเพิ่มเติมลงไปดังนี้

```
<note>
  <date>1999-08-01</date>
  <to>Tove</to>
  <from>Jani</from>
  <heading>Reminder</heading>
  <body>Don't forget me this weekend!</body>
</note>
```

เมื่อเอกสาร XML เป็นดังนี้แล้ว แอปพลิเคชันที่สร้างไว้จะไม่เสียหาย เนื่องจาก แอปพลิเคชันจะยังคงค้นหา element to, from และ heading ในเอกสาร XML และยังคงให้ผลลัพธ์เหมือนเดิม

■ XML Element มีความสัมพันธ์กัน

ในการที่จะเข้าใจถึงภาพรวมของ XML จะต้องเข้าใจถึงความสัมพันธ์ระหว่างชื่อของ XML element ที่ตั้งและ content ของ element

คำอธิบายของหนังสือดังนี้

Book Title: My First XML

Chapter 1: Introduction to XML

What is HTML

What is XML

Chapter 2: XML Syntax

Elements must have a closing tag

Elements must be correctly nested

สามารถมีเอกสาร XML ที่อธิบายหนังสือได้นั้น ได้ดังนี้

```
<book>
  <title>My First XML</title>
  <prod id="33-657" media="paper"></prod>
  <chapter>Introduction to XML
    <para>What is HTML</para>
    <para>What is XML</para>
  </chapter>
  <chapter>XML Syntax
    <para>Elements must have a closing tag</para>
    <para>Elements must be properly nested</para>
  </chapter>
</book>
```

book เป็น root element title และ chapter เป็น child element ของ book book เป็น parent element ของทั้ง title และ chapter title และ chapter จึงเป็น sibling กันเนื่องจากมี parent เดียวกัน

■ Element มี Content

XML element คือทั้งหมด ตั้งแต่เปิดแท็กจนถึงปิดแท็ก element หนึ่งสามารถมี element content, mixed content, simple content หรือ empty content และยังมี attribute ได้ด้วย

จากตัวอย่างข้างบน book มี element content เพราะมันประกอบด้วย element อื่น chapter เป็น mixed content เพราะมันประกอบด้วยแท็กและ element อื่น ๆ para เป็น simple content (text content) เพราะมันประกอบด้วยแท็กเท่านั้น prod เป็น empty element เนื่องจากมันไม่มีข้อมูลใดๆ และเฉพาะ prod element เท่านั้นที่มี attribute attribute ที่ชื่อ id มีค่าเท่ากับ “33-657” attribute ที่ชื่อ media มีค่าเท่ากับ “paper”

■ การตั้งชื่อ Element

XML element มีกฎการตั้งชื่อดังนี้

- ชื่อประกอบด้วย ตัวอักษร , ตัวเลข และอักขระอื่นๆ
- ชื่อต้องไม่ขึ้นต้นด้วยตัวเลขหรือ “_” (underscore)
- ชื่อต้องไม่ขึ้นด้วยคำว่า XML (หรือ Xml หรือ xml)
- ชื่อไม่สามารถมีช่องว่างได้ (space)

- ชื่อต้องไม่ควรประกอบด้วย “.” เพราะมันถูกสงวนไว้กับการใช้ namespace

ทุกๆชื่อสามารถใช้ได้ ไม่มีคำใดเป็นคำสงวน แต่จะเป็นการดีในการที่จะหลีกเลี่ยงการใช้ “-“ หรือ “.” แล้วใช้เฉพาะ “_” เท่านั้น เนื่องจากอาจเกิดการสับสนของซอฟต์แวร์ในกรณีพยายามที่จะตัดคำ (first-name) หรือคิดว่า “name” เป็นคุณสมบัติของ object “first” (first.name)

3.1.4 XML Attribute

XML attribute มีคุณสมบัติดังนี้

- รูปแบบของ Quote , “female” or ‘female’

ค่าของ attribute ต้องอยู่ในเครื่องหมายคำพูดเสมอ แต่อาจเป็น ‘ (single quote) หรือ “ (double quote) ก็ได้

<person sex=”female”>	✓
หรือ	
<person sex=’female’>	✓

Double quote จะเป็นที่ใช้โดยทั่วไปมากกว่า แต่ในบางครั้ง (ถ้าค่าของ attribute จำเป็นที่จะต้องมี quote) จึงจำเป็นที่จะต้องใช้ single quote ดังนี้

<gangster name=’George “Shotgun” Ziegler’>
--

- การใช้ Element และ Attribute

ข้อมูลสามารถที่จะถูกเก็บใน child element หรือ attribute ดังนี้

<person sex=”female”>
<firstname>Anna</firstname>
<lastname>Smith</lastname>
</person>

หรือ

<person>
<sex>female</sex>
<firstname>Anna</firstname>
<lastname>Smith</lastname>
</person>

ตัวอย่างแรก sex เป็น attribute ตัวอย่างที่ 2 sex เป็น child element ซึ่งทั้งสองตัวอย่างให้ความหมายเดียวกัน มันไม่มีกฎว่าจะใช้ attribute เมื่อไร หรือ child element เมื่อไร แต่แนะนำว่าควรหลีกเลี่ยงการใช้ attribute ควรจะใช้ child element ถ้าอินฟอร์เมชันนั้นดูเหมือนจะเป็นข้อมูล (data)

■ หลีกเลี่ยงการใช้ Attribute

บางปัญหาของการใช้ attribute

- attribute ไม่สามารถมีหลายค่า (child element สามารถทำได้)
- attribute ไม่ง่ายที่จะขยาย (ในอนาคต)
- attribute ไม่สามารถอธิบายโครงสร้าง (child element สามารถทำได้)
- attribute ยากที่จะจัดการด้วยโค้ดของโปรแกรม
- ค่าของ attribute ไม่ง่ายในการจะตรวจสอบกับ DTD ที่มีอยู่

แต่อย่างไรก็ตาม จะสามารถใช้ attribute ก็ต่อเมื่อ อินฟอร์เมชันนั้นไม่สัมพันธ์กับข้อมูล ตัวอย่างเช่น ในบางครั้ง ID reference สามารถถูกใช้ในการเข้าถึง XML element

```
<messages>

  <note ID="501">
    <to>Tove</to>
    <from>Jani</from>
    <heading>Reminder</heading>
    <body>Don't forget me this weekend!</body>
  </note>

  <note ID="502">
    <to>Jani</to>
    <from>Tove</from>
    <heading>Re: Reminder</heading>
    <body>I will not!</body>
  </note>

</messages>
```

ID ในตัวอย่างนี้เป็น counter หรือ unique identifier ในการอ้างถึง note ที่ต่างกันของไฟล์ XML ไม่ใช่ส่วนหนึ่งของข้อมูล note

3.1.5 XML Validation

เอกสาร XML แบ่งการตรวจสอบความถูกต้องของเอกสารเป็น 2 ชนิดคือ

1) “Well Formed” XML Document

“Well Formed” XML document เป็นเอกสารที่เป็นไปตามกฎไวยากรณ์ XML ที่ได้กล่าวมาข้างต้น

2) “Valid” XML Document

“Valid” XML document เป็นเอกสาร XML ที่มีรูปแบบที่เป็นไปตามกฎของ DTD โดยจะต้องตรงตามโครงสร้างข้อมูลของเอกสารที่ได้ประกาศโดย DTD

■ XML DTD

วัตถุประสงค์ของ DTD คือกำหนดโครงสร้างของเอกสาร XML เพื่อใช้ในการตรวจสอบว่าเอกสาร XML นั้นถูกต้องตามรูปแบบหรือไม่ เช่น element BOOK จะต้องมีการมี element NAME เสมอ อาจจะมี element PAGE หรือไม่ก็ได้ เป็นต้น

■ XML Schema

XML schema มีวัตถุประสงค์เหมือนกับ DTD จะถูกนำมาใช้แทน DTD เนื่องจาก

- ง่ายในการเรียนรู้มากกว่า DTD
- ใช้ได้ดีกว่า DTD
- schema เขียนด้วย XML
- รองรับ datatype
- รองรับ namespace

3.1.6 แนะนำ DTD

1) การประกาศ DOCTYPE ภายใน

ถ้า DTD ถูกรวมอยู่ในไฟล์ XML นั้น มันจะต้องอยู่ภายในการให้คำจำกัดความ DOCTYPE ตามไวยากรณ์นี้

```
<!DOCTYPE root-element [element-declarations]>
```

ตัวอย่างดังนี้

```
<?XML version="1.0"?>
<!DOCTYPE note [
  <!ELEMENT note (to,from,heading,body)>
  <!ELEMENT to (#PCDATA)>
```

```

<!ELEMENT from (#PCDATA)>
<!ELEMENT heading (#PCDATA)>
<!ELEMENT body (#PCDATA)>
]>
<note>
  <to>Tove</to>
  <from>Jani</from>
  <heading>Reminder</heading>
  <body>Don't forget me this weekend</body>
</note>

```

DTD ข้างบนนี้สามารถแปลความหมายได้ดังนี้

!DOCTYPE note (บรรทัดที่ 2) กำหนดว่าเป็นเอกสารของ type note

!ELEMENT note (บรรทัดที่ 3) กำหนดว่า note element มี 4 element คือ to , from , heading และ body

!ELEMENT to (บรรทัดที่ 4) กำหนดว่า to element เป็นข้อมูลชนิด #PCDATA

!ELEMENT from (บรรทัดที่ 5) กำหนดว่า from element เป็นข้อมูลชนิด #PCDATA

2) การประกาศ DOCTYPE ภายนอก

ถ้า DTD อยู่ภายนอกไฟล์ XML มันจะต้องอยู่ภายในการให้คำจำกัดความ DOCTYPE ตามไวยากรณ์นี้

```
<!DOCTYPE root-element SYSTEM "filename">
```

ตัวอย่างไฟล์ XML เป็นดังนี้

```

<?XML version="1.0"?>
<!DOCTYPE note SYSTEM "note.dtd">
<note>
  <to>Tove</to>
  <from>Jani</from>
  <heading>Reminder</heading>
  <body>Don't forget me this weekend!</body>
</note>

```

ไฟล์ของ DTD ชื่อ note.dtd เป็นดังนี้

note.dtd

```
<!ELEMENT note (to,from,heading,body)>
<!ELEMENT to (#PCDATA)>
<!ELEMENT from (#PCDATA)>
<!ELEMENT heading (#PCDATA)>
<!ELEMENT body (#PCDATA)>
```

3.1.6.1 โครงสร้างของ XML

เอกสาร XML จะถูกสร้างจากส่วนต่างๆ ดังนี้

- Element – เป็นโครงสร้างหลักของ XML
- Tag - ใช้ในการกำหนด markup element
- Attribute - ให้ข้อมูลเพิ่มเติมกับ element
- Entity – เป็นตัวแปรที่ใช้ในการกำหนดเท็กซ์ทั่ว ๆ ไป entity reference เป็นการอ้างถึง entity เช่น entity ชื่อ domain มีค่าเท่ากับ www.w3.org entity reference คือ &domain หากภายในเอกสาร XML มี &domain ค่าของมันจะมีค่าเท่ากับ www.w3.org
- PCDATA - หมายถึง parse character data PCDATA เป็นเท็กซ์ที่จะถูก parse โดย parser
- CDATA - หมายถึง character data เป็นเท็กซ์ที่จะไม่ถูก parse โดย parser

3.1.6.2 การกำหนด Element ใน DTD

วิธีการในการประกาศ element จะเป็นดังนี้

- การประกาศ Element
- ประกาศตามไวยากรณ์นี้

```
<!ELEMENT element-name category>
```

หรือ

```
<!ELEMENT element-name (element-content)>
```

- Element ว่างเปล่า (Empty)
- ถูกประกาศโดยคีย์เวิร์ด EMPTY

```
<!ELEMENT element-name EMPTY>
```

DTD example:

```
<!ELEMENT br EMPTY>
```

XML example:

```
<br />
```

- **Element ที่เป็นข้อมูลชนิดตัวอักษร**

element ที่เป็นตัวอักษรเท่านั้นจะถูกประกาศด้วยคีย์เวิร์ด PCDATA ที่เปิดและปิดด้วย “(“ ”)”

```
<!ELEMENT element-name (#PCDATA)>
```

DTD example:

```
<!ELEMENT note (#PCDATA)>
```

- **Element ที่เป็นข้อมูลใด ๆ**

ประกาศกับคีย์เวิร์ด ANY content จะสามารถเป็นข้อมูลชนิดใดก็ได้

```
<!ELEMENT element-name ANY>
```

DTD example:

```
<!ELEMENT note ANY>
```

- **Element และ Children (Sequences)**

ประกาศโดยใช้ “,” ในการแยก child element แล้วเปิดและปิดด้วย “(“ ”)”

```
<!ELEMENT element-name (child-element-name)>
```

หรือ

```
<!ELEMENT element-name (child-element-name,child-element-name,.....)>
```

DTD example:

```
<!ELEMENT note (to,from,heading,body)>
```

ถ้ามีการประกาศ child element จะต้องประกาศ child element ให้สมบูรณ์ด้วยดังตัวอย่างข้างล่างนี้

```
<!ELEMENT note (to,from,heading,body)>
```

```
<!ELEMENT to (#PCDATA)>
<!ELEMENT from (#PCDATA)>
<!ELEMENT heading (#PCDATA)>
<!ELEMENT body (#PCDATA)>
```

- การประกาศเพียงครั้งเดียวของ Element เดียวกัน

```
<!ELEMENT element-name (child-name)>
```

DTD example:

```
<!ELEMENT note (message)>
```

จากตัวอย่างหมายความว่า note element จะประกอบด้วย child element message เท่านั้น

- การประกาศอย่างน้อย 1 ครั้งของ Element เดียวกัน

```
<!ELEMENT element-name (child-name+)>
```

DTD example:

```
<!ELEMENT note (message+)>
```

เครื่องหมาย + ถูกใช้ในความหมายหนึ่งหรือมากกว่า จึงหมายความว่า note element จะประกอบด้วย child element message ตั้งแต่ 1 ค่าขึ้นไป

- การไม่ประกาศหรือประกาศเท่าไรก็ได้ของ Element เดียวกัน

```
<!ELEMENT element-name (child-name*)>
```

DTD example:

```
<!ELEMENT note (message*)>
```

เครื่องหมาย * ถูกใช้ในความหมายว่าไม่มีหรือมีเท่าไรก็ได้ จึงหมายความว่า note element จะประกอบด้วย child element message ก็ค่าก็ได้ หรือไม่มีเลยก็ได้

- การไม่ประกาศหรือประกาศเพียงครั้งเดียวของ Element เดียวกัน

```
<!ELEMENT element-name (child-name?)>
```

DTD example:

```
<!ELEMENT note (message?)>
```

เครื่องหมาย ? ถูกใช้ในความหมาย ไม่มีหรือมีเพียงหนึ่ง จึงหมายความว่า note element จะประกอบด้วย child element message หนึ่งค่า หรือไม่มีก็ได้

- การประกาศ Content แบบให้เลือก

จะใช้เครื่องหมาย | แทนความหมายหรือ

DTD example:

```
<!ELEMENT note (to,from,header,message|body)>
```

ดังนั้นตัวอย่างนี้จึงหมายถึง note element จะต้องประกอบด้วย to element , from element , header element แล้วก็ message element หรือ body element อย่างใดอย่างหนึ่งเท่านั้น

- การประกาศ Content แบบผสม

เป็นการผสมการประกาศหลาย ๆ อย่างเข้าด้วยกัน

DTD example:

```
<!ELEMENT note (#PCDATA|to|from|header|message)*>
```

ตัวอย่างนี้หมายถึง note element จะประกอบด้วย character data หรือ element to, from, header และ message จำนวนเท่าไรก็ได้

3.1.6.3 การกำหนด Attribute ใน DTD

วิธีการประกาศ attribute จะเป็นดังนี้

■ การประกาศ Attribute

attribute ต้องถูกประกาศตามไวยากรณ์ นี้

```
<!ATTLIST element-name attribute-name attribute-type default-value>
```

DTD example:

```
<!ATTLIST payment type CDATA "check">
```

XML example:

```
<payment type="check">
```

ชนิดของ attribute สามารถมีค่าได้ดังตารางที่ 3-1

ค่า	คำอธิบาย
CDATA	ค่าต้องเป็น character ที่จะไม่ถูก parse โดย parser
(en1 en2 ..)	ค่าต้องเป็นอันใดอันหนึ่งในรายการนั้น
ID	ค่าต้องเป็น unique id
IDREF	ค่าต้องเป็น id ของ element อื่น
IDREFS	ค่าต้องเป็นรายการของ id อื่น โดยจะเว้นแต่ละรายการด้วยช่องว่าง
NMTOKEN	ค่าต้องเป็นชื่อที่ประกอบด้วยตัวอักษร , ตัวเลข , เครื่องหมายจุด (.) , เครื่องหมายขีดกลาง (-) , เครื่องหมายขีดล่าง (_) และสามารถมีเครื่องหมายโคลอน (:) ยกเว้นในตำแหน่งแรกได้
NMTOKENS	ค่าต้องเป็นรายการของ NMTOKEN โดยจะเว้นแต่ละรายการด้วยช่องว่าง
ENTITY	ค่าต้องเป็น entity
ENTITIES	ค่าต้องเป็นรายการของ entity
NOTATION	ค่าต้องเป็นชื่อของ notation (notation คือชนิดของข้อมูลภายนอกที่กำหนดขึ้นเอง เช่น GIF เพื่อบอกว่าเป็นไฟล์รูปภาพ)
XML:	ค่าต้องขึ้นต้นด้วย XML

ตารางที่ 3-1 ชนิดของ Attribute ใน XML

default-value สามารถมีค่าได้ดังตารางที่ 3-2

ค่า	คำอธิบาย
#DEFAULT value	ค่าดีฟอลต์ของ attribute
#REQUIRED	element ต้องมี attribute
#IMPLIED	element จะมี attribute หรือไม่ก็ได้
#FIXED value	attribute ต้องเหมือนกับที่กำหนดไว้

ตารางที่ 3-2 ค่าดีฟอลต์ของ Attribute ใน XML

■ ตัวอย่างของการประกาศ Attribute

DTD example:

```
<!ELEMENT square EMPTY>
```

```
<!ATTLIST square width CDATA "0">
```

XML example:

```
<square width="100"></square>
```

ตัวอย่างนี้หมายความว่า square element เป็น empty element กับ attribute width ซึ่งเป็นชนิด CDATA ถ้าไม่ระบุจะยึดค่า default เป็น 0

■ ค่าของ Attribute แบบดีฟอลต์

```
<!ATTLIST element-name attribute-name attribute-type "default-value">
```

DTD example:

```
<!ATTLIST payment type CDATA "check">
```

XML example:

```
<payment type="check">
```

ถ้าในเอกสาร XML ไม่มีการระบุจะใช้ค่าดีฟอลต์ แต่ถ้าระบุจะใช้ตามที่ระบุไว้

■ Attribute โดยนัย

```
<!ATTLIST element-name attribute-name attribute-type #IMPLIED>
```

DTD example:

```
<!ATTLIST contact fax CDATA #IMPLIED>
```

XML example:

```
<contact fax="555-667788">
```

จะใช้ #IMPLIED ในกรณีจะระบุ attribute หรือ ไม่ได้ก็ได้ ถ้าไม่ระบุก็จะไม่มีค่า default ให้

■ Attribute ที่ต้องระบุเสมอ

```
<!ATTLIST element-name attribute_name attribute-type #REQUIRED>
```

DTD example:

```
<!ATTLIST person number CDATA #REQUIRED>
```

XML example:

```
<person number="5677">
```

ใช้ในการบังคับให้ต้องระบุ attribute ถ้าในเอกสาร XML ไม่ระบุ attribute เอกสารนั้นจะไม่ถูกต้อง

■ ค่าของ Attribute คงที่

```
<!ATTLIST element-name attribute-name attribute-type #FIXED "value">
```

DTD example:

```
<!ATTLIST sender company CDATA #FIXED "Microsoft">
```

XML example:

```
<sender company="Microsoft">
```

ในแบบนี้ถ้าไม่ได้ระบุค่า attribute จะใช้ค่าดีฟอลต์ แต่ถ้าจะระบุต้องระบุให้ตรงกับค่าดีฟอลต์ มิฉะนั้นจะถือว่าผิด

- ค่าของ Attribute กำหนดเอง

```
<!ATTLIST element-name attribute-name (en1|en2|..) default-value>
```

DTD example:

```
<!ATTLIST payment type (check|cash) "cash">
```

XML example:

```
<payment type="check">
```

หรือ

```
<payment type="cash">
```

จะใช้เมื่อเราต้องการกำหนดค่าของ attribute เอง

3.1.6.4 การกำหนด Entity ใน DTD

- การประกาศ Entity ภายใน

```
<!ENTITY entity-name "entity-value">
```

DTD Example:

```
<!ENTITY writer "Donald Duck.">
```

```
<!ENTITY copyright "Copyright W3Schools.">
```

XML example:

```
<author>&writer;&copyright;</author>
```

- การประกาศ Entity ภายนอก

```
<!ENTITY entity-name SYSTEM "URI/URL">
```

DTD Example:

```
<!ENTITY writer
```

```
SYSTEM "http://www.w3schools.com/entities/entities.XML">
```

```
<!ENTITY copyright
```

```
SYSTEM "http://www.w3schools.com/entities/entities.dtd">
```

XML example:

```
<author>&writer;&copyright;</author>
```

3.1.7 XML Namespace

XML namespace จะหาวิธีหลีกเลี่ยงความสับสนที่เกิดจากชื่อ element (Name Conflicts)

■ ความสับสนที่เกิดจากชื่อ

ความสับสนที่เกิดจากชื่อ element จะเกิดขึ้นบ่อยเมื่อเอกสารที่แตกต่างกัน ใช้ชื่อเดียวกันในการอธิบายชนิดของ element ที่แตกต่างกัน เช่น table ใน 2 ตัวอย่างต่อไปนี้

```
<table>
  <tr>
    <td>Apples</td>
    <td>Bananas</td>
  </tr>
</table>
```

และ

```
<table>
  <name>African Coffee Table</name>
  <width>80</width>
  <length>120</length>
</table>
```

ถ้านำเอกสาร XML ทั้ง 2 มารวมกัน จะเกิดความสับสนจากชื่อขึ้นเพราะทั้ง 2 มี element (table) ซึ่งมีใช้ในความหมายที่ต่างกัน

■ การแก้ปัญหาความสับสนที่เกิดจากชื่อโดยใช้ Prefix

เอกสาร XML ทั้ง 2 คือ

```
<h:table>
  <h:tr>
```

```
<h:td>Apples</h:td>
<h:td>Bananas</h:td>
</h:tr>
</h:table>
```

และ

```
<f:table>
  <f:name>African Coffee Table</f:name>
  <f:width>80</f:width>
  <f:length>120</f:length>
</f:table>
```

ความสับสนที่เกิดจากชื่อจะถูกกำจัดไปโดยการเพิ่ม prefix ทำให้ได้ element ที่ต่างกัน 2 ชนิด (<h:table> และ <f:table>)

■ การใช้ Namespaces

จากเอกสาร XML ทั้ง 2 คือ

```
<h:table xmlns:h="http://www.w3.org/TR/html4/">
  <h:tr>
    <h:td>Apples</h:td>
    <h:td>Bananas</h:td>
  </h:tr>
</h:table>
```

และ

```
<f:table xmlns:f="http://www.w3schools.com/furniture">
  <f:name>African Coffee Table</f:name>
  <f:width>80</f:width>
  <f:length>120</f:length>
</f:table>
```

เราสามารถเพิ่ม attribute xmlns ลงในแท็ก เพื่อบอกชื่อที่ระบุไว้ ที่เกี่ยวข้องกับ namespace

■ Attribute Namespace

attribute namespace จะวางในแท็กเริ่มต้น (start tag) ตามไวยากรณ์ ดังนี้

```
Xmlns:namespace-prefix="namespace"
```

จากตัวอย่างข้างบน namespace จะสามารถกำหนดด้วย Internet address

```
Xmlns:f="http://www.w3schools.com/furniture">
```

จาก namespace specification ของ W3C กล่าวว่า namespace สามารถเป็น Uniform Resource Identifier (URI)

ถ้า namespace ถูกกำหนดในแท็กเริ่มต้นแล้ว child element ทุกตัวที่มี prefix เหมือนกันจะต้อง มีความสัมพันธ์กับ namespace เดียวกันด้วย

และ address ที่ใช้บอก namespace ใช้เพื่อให้ชื่อมีความเป็นเอกลักษณ์ไม่ได้ใช้เป็น address เพื่อ อ้างอิงหาข้อมูล แต่หลายบริษัทใช้ namespace เพื่อเป็นตัวชี้ไปยัง web page ที่มีอยู่จริงซึ่งเก็บข้อมูล เกี่ยวกับ namespace ไว้

■ Default Namespaces

การกำหนด default namespace จะป้องกันการใช้ prefix ใน child element ทุกตัว มีไวยากรณ์ ดังนี้

```
<element xmlns="namespace">
```

ซึ่งจากตัวอย่างที่ผ่านมาจะได้ดังนี้

```
<table xmlns="http://www.w3.org/TR/html4/">
  <tr>
    <td>Apples</td>
    <td>Bananas</td>
  </tr>
</table>
```

และ

```
<table xmlns="http://www.w3schools.com/furniture">
  <name>African Coffee Table</name>
  <width>80</width>
```

```
<length>120</length>
</table>
```

3.1.8 XML PCDATA and CDATA

Parsable Character Data (PCDATA) คือข้อมูลแบบเท็กซ์ที่จะถูก parse โดย parser

Character Data (CDATA) คือข้อมูลแบบเท็กซ์ที่ไม่ถูก parse โดย parser

3.1.8.1 PCDATA

- **XML Parser จะมองทุกเท็กซ์เป็น Parsable Characters (PCDATA)**

เมื่อ XML ถูก parse เท็กซ์ที่อยู่ระหว่าง XML แท็กจะถูก parse ไปด้วย

```
<message>This text is also parsed</message>
```

parser ทำเช่นนี้เนื่องจาก XML element สามารถมีหลาย element อยู่ภายใน ดังในตัวอย่าง

```
<name><first>Bill</first><last>Gates</last></name>
```

และ parser จะแบ่งเป็น sub-element ดังนี้

```
<name>
  <first>Bill</first>
  <last>Gates</last>
</name>
```

- **Escape Character**

XML character ที่ไม่สามารถใช้ได้จะถูกแทนที่ด้วย entity reference

ถ้าใส่ตัวอักษร "<" ใน XML element จะทำให้เกิดความผิดพลาด เพราะ parser จะแปลเป็นจุดเริ่มต้นของ element ใหม่ ดังนั้นจึงไม่สามารถเขียน อย่างนี้ได้

```
<message>if salary < 1000 then</message>
```

การหลีกเลี่ยง สามารถทำได้โดย แทนที่ "<" ด้วย entity reference ดังนี้

```
<message>if salary &lt; 1000 then</message>
```

entity reference มาตรฐานที่ได้กำหนดมาให้ก่อนแล้วใน XML มีทั้งหมด 5 ตัว ดังตารางที่ 3-3

Entity Reference	ค่า	ความหมาย
<	<	less than
>	>	greater than
&	&	ampersand
'	'	apostrophe
"	"	quotation mark

ตารางที่ 3-3 Entity Reference มาตรฐาน

entity reference จะขึ้นต้นด้วย & และลงท้ายด้วย ;

เฉพาะ “<” และ “&” เท่านั้นที่ไม่สามารถใช้ได้ใน XML ส่วนตัวที่เหลือควรทำให้เป็นนิสย

3.1.8.2 CDATA

parser จะไม่ parse ทุกอย่างที่อยู่ในส่วนของ CDATA

ถ้าเท็กซ์มี “<” หรือ “&” อยู่ภายใน XML element นั้นจะถูกกำหนดเป็นส่วนของ CDATA ส่วนของ CDATA จะเริ่มต้นด้วย “<![CDATA[“ และลงท้ายด้วย “]]>”

```

<script>
<![CDATA[
function matchwo(a,b)
{
if (a < b && a < 0) then
{
return 1
}
else
{
return 0
}
}
]]>
</script>

```

จากตัวอย่างข้างต้น ทุกอย่างที่อยู่ในส่วนของ CDATA จะถูกเพิกเฉยโดย parser

3.1.9 XML Parser

XML parser คือโปรแกรมที่ใช้ในการอ่าน สร้าง และจัดการกับเอกสาร XML รวมถึงตรวจสอบความถูกต้องของเอกสาร XML parser สามารถเขียนได้ด้วยภาษาใด ๆ ก็ได้ ในบราวเซอร์ IE 5.0 ขึ้นไปก็มี Microsoft XML parser ทำให้ IE สามารถแสดงผลเอกสาร XML ได้ XML parser ของภาษาจาวา อย่างเช่น XML parser ของซัน (<http://java.sun.com/xml>) หากเราต้องการสร้างแอปพลิเคชันที่สามารถจัดการกับ XML เราก็ต้องมี parser ของภาษานั้นและเขียนโค้ดติดต่อกับ API ของมัน

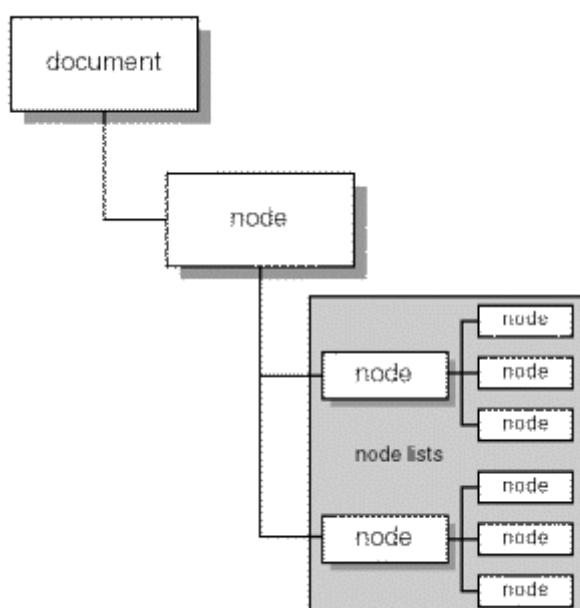
XML parser มี 2 ชนิดคือ

- DOM (Document Object Model)
- SAX (Simple API for XML)

3.1.9.1 DOM

DOM จะใช้วิธี parse เอกสาร XML เป็นลำดับชั้น (hierarchy) ของออบเจกต์ดังนี้ (ดังรูปที่ 3-1)

- 1) Document Object – แหล่งข้อมูล XML
- 2) Node Object – Parent node ของ child node
- 3) Node List Object – list ของ sibling node
- 4) Parse Error – ใช้ในการรับค่าผิดพลาดที่เกิดจากการ parse

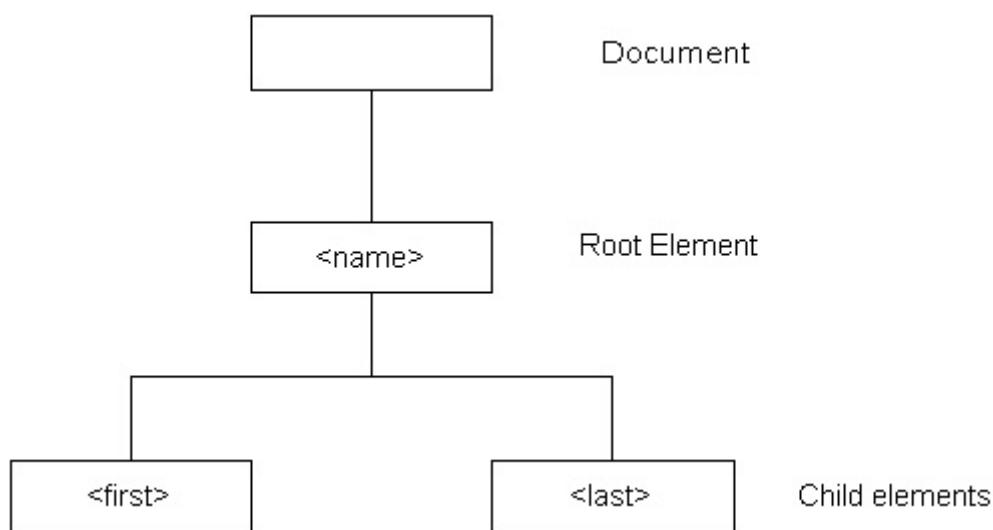


รูปที่ 3-1 โครงสร้างของ DOM

หากเอกสาร XML เป็นดังนี้

```
<name>
  <first>Jim(</first>
  <last>Jones</last>
</name>
```

จะได้ความสัมพันธ์ดังรูปที่ 3-2



รูปที่ 3-2 ความสัมพันธ์ของการ parse ด้วย DOM

3.1.9.2 SAX

SAX (Simple API for XML) วิธีนี้จะเป็นแบบ event-driven โดย event จะประกอบด้วย

- 1) Start Element – จะตอบสนองกับ event นี้เมื่อพบจุดเริ่มต้นของแท็ก XML
- 2) End element – จะตอบสนองกับ event นี้เมื่อพบจุดสิ้นสุดของแท็ก
- 3) Character – จะตอบสนองกับ event นี้เมื่อพบกับตัวอักษร
- 4) Start Document – จะตอบสนองกับ event นี้ในตอนเริ่ม parse
- 5) End Document - จะตอบสนองกับ event นี้ในเมื่อจบการ parse

ดังนั้นจากโค้ดข้างบนจะเกิด event ดังนี้

	→	Start Document
<name>	→	Start Element name
<first>	→	Start Element first

Jim	→	Character Jim
</first>	→	End Element first
<last>	☐	Start Element last
Jones	☐	Character Jones
</last>	☐	End Element last
</name>	☐	Start Element name
	☐	End Document

ทั้ง 2 วิธีมีข้อดีข้อเสียต่างกันคือ DOM จะใช้เมโมรีและมีการประมวลผลมาก แต่สามารถจัดการได้ง่าย สามารถที่จะแก้ไขค่าได้ ไม่เหมาะกับเอกสาร XML ที่มีขนาดใหญ่ ถูกนำไปใช้ในบราวเซอร์ ในขณะที่ SAX ไม่ต้อง parse เอกสาร XML ทั้งหมดจึงใช้เมโมรีและมีการประมวลผลน้อยกว่า เหมาะกับการสร้างและรับค่าจากเอกสาร จะถูกใช้ในการประมวลผลทางฝั่งเซิร์ฟเวอร์ ตัวอย่าง XML parser ดังตารางที่ 3-4

ชื่อ	แพลตฟอร์ม	คำบรรยาย
ActiveDOM	Microsoft Window	http://www.vivid-creations.com
ActiveSAX	Microsoft Window	http://www.vivid-creations.com
JavaTM API for XML	Java	http://java.sun.com/xml
MS XML	Microsoft Window	http://www.microsoft.com
Oracle XML Developer's Kit	Java , C++ , PL/SQL	http://technet.oracle.com/tech/xml
Xerces	Java , C++	http://xml.apache.org
XP	Java	http://www.jclark.com/xml

ตารางที่ 3-4 XML Parser

3.1.10 ประโยชน์จาก XML

สำหรับประโยชน์ของ XML นั้น เป็นด้านความยืดหยุ่นในการใช้งานสำหรับแอปพลิเคชันที่อิงกับ Web Base ที่ง่ายในการค้นหาข้อมูล มีความยืดหยุ่นในการพัฒนาเว็บ สามารถผสมผสานข้อมูลจากหลายแหล่ง จากแอปพลิเคชันที่ต่างกัน สามารถแสดงข้อมูลแบบต่างๆ และสามารถ update ข้อมูลให้ทันสมัยเสมอ และคาดว่าจะเป็มาตรฐานใหม่ของระบบเปิด ซึ่งนับเป็น format ใหม่สำหรับการส่งข้อมูลบนเว็บที่มากด้วยข้อมูลหลายแบบ แต่ส่งผ่านด้วยเทคโนโลยีที่บีบอัดข้อมูล ที่ให้ความเร็วได้รับการสนับสนุนจากผลิตภัณฑ์ค่ายไมโครซอฟท์

สถาปัตยกรรม XML

ภาษา XML ได้รับการสนับสนุนจาก W3C ให้นักพัฒนาเว็บแอปพลิเคชันได้หันมาใช้เป็นส่วนประกอบของการพัฒนาเว็บไซต์ เพราะ XML มีประสิทธิภาพและมีความน่าเชื่อถือสูงในการแปลงข้อมูลและโครงสร้างข้อมูลให้สามารถนำไปใช้งาน สถาปัตยกรรม 3 Tier ที่ XML สามารถสร้างขึ้นจากระบบข้อมูลที่ใช้โมเดลของ 3-Tier โครงสร้าง ของข้อมูลต่างๆ สามารถนำมาแสดงตามข้อกำหนด หรือรูปแบบที่ต้องการตามการใช้งานได้

XML เป็นการทำงานในระดับกลาง middle tier ที่สามารถเรียกใช้ฐานข้อมูลได้หลากหลายระบบ ฐานข้อมูลและโอนข้อมูลให้อยู่ในรูปแบบของ XML และมีการให้รายละเอียดเกี่ยวกับตัวข้อมูล โครงสร้างต่างๆ ของระบบฐานข้อมูลได้ XML เป็นระบบเปิดที่นำเสนอข้อมูลในรูปแบบ text ผ่านทาง HTTP เหมือนกับ HTML แต่จะมีคุณสมบัติในการให้ข้อมูลแบบ real time อัปเดตหรือเปลี่ยนแปลงได้ตามข้อกำหนด การแสดงข้อมูลจาก XML ใน HTML จะเป็นการเพิ่มในส่วนของการรายละเอียดข้อมูล ที่มีการเรียกใช้จากแหล่งหรือฐานข้อมูลที่เชื่อมโยงกันในหลายแหล่งเพื่อให้ HTML มีความสมบูรณ์มากขึ้น ในอนาคตการพัฒนาเว็บหรือการเขียน และสร้าง HTML ไม่จำเป็นต้องมีการเขียนชุดคำสั่งที่ยุ่งยากซับซ้อน มากก็สามารถทำงานร่วมกับระบบข้อมูลได้อย่างมีประสิทธิภาพ XML จะทำการกำหนดค่าสำหรับ โครงสร้างข้อมูลที่จะนำไปแสดงใน HTML นอกจากนั้นยังสามารถนำไปสนับสนุนระบบการแลกเปลี่ยน ข้อมูลข่าวสารทาง Electronic ได้อย่างดีอีกด้วย

3.1.11 บทสรุปของ XML

XML มีความยืดหยุ่นพอในการนำเสนอข้อมูลแบบต่างๆ ที่สามารถให้รายละเอียดโครงสร้าง ข้อมูลตามระดับและความต้องการในการนำไปใช้งาน

3.2 SOAP

3.2.1 แนะนำ SOAP

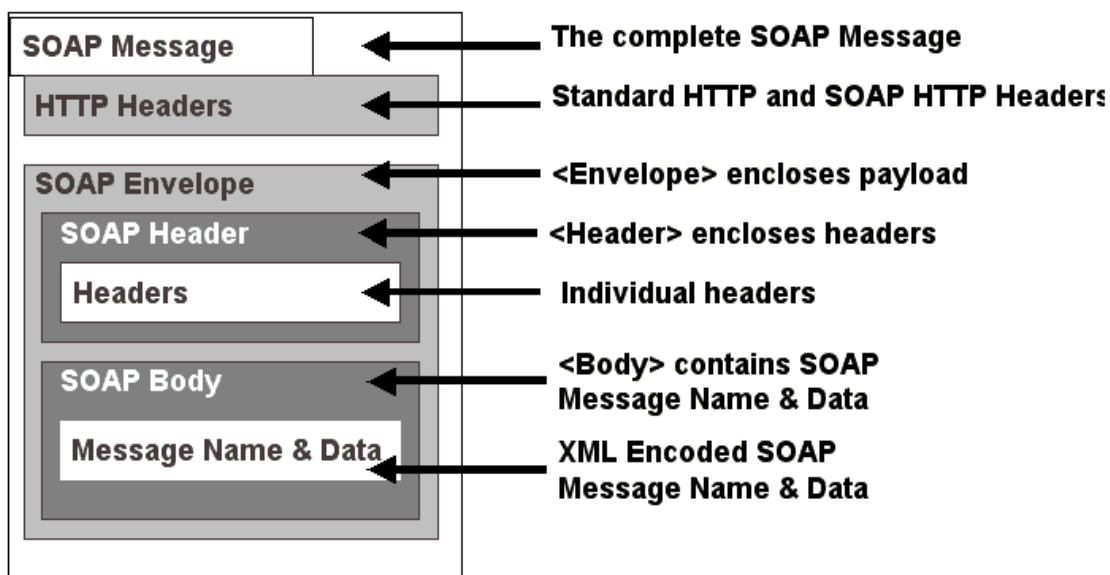
SOAP (Simple Object Access Protocol) เป็นโพรโทคอลที่ใช้ XML เป็นพื้นฐาน เพื่อให้ซอฟต์แวร์คอมพิวเตอร์ และแอปพลิเคชันสามารถติดต่อกันผ่าน HTTP ซึ่งเป็นมาตรฐานอินเทอร์เน็ต โพรโทคอลได้

เนื่องจากการสื่อสารระหว่างแอปพลิเคชันจำเป็นต้องการพัฒนาแอปพลิเคชันบนอินเทอร์เน็ต เป็นอย่างมาก แต่แอปพลิเคชันแบบกระจายกันทำงาน (distributed application) ในปัจจุบันใช้ Remote Procedure Call (RPC) สื่อสารกันระหว่างออบเจกต์ เช่น DCOM และ CORBA ซึ่งไม่ได้ใช้พอร์ตเดียวกับ HTTP ที่เป็นพอร์ตมาตรฐานสำหรับให้บริการเว็บ ดังนั้น RPC จึงนำมาปรับใช้กับอินเทอร์เน็ตได้ยาก และมีปัญหาทางด้านความปลอดภัย ไฟร์วอลล์และพร็อกซีเซิร์ฟเวอร์จะไม่ยอมให้ส่งข้อมูลชนิดนี้ได้ตามปกติ วิธีที่ดีกว่าคือใช้ HTTP เพราะเป็นที่ยอมรับโดยอินเทอร์เน็ตบราวเซอร์ และเซิร์ฟเวอร์ทุกชนิด ซึ่ง SOAP ถูกสร้างมาเพื่อใช้ในกรณีนี้

ข้อดีของ SOAP ก็คือ SOAP ไม่ขึ้นกับคอมพิวเตอร์เทคโนโลยี และภาษาการเขียนโปรแกรมใดๆ สามารถเขียนได้ง่ายและขยายเพิ่มเติมได้

3.2.2 ส่วนประกอบของ SOAP

SOAP เมสเสจใช้ไวยากรณ์ของ XML ในการสร้าง ประกอบด้วย 3 อีลีเมนต์มาตรฐาน คือ SOAP Envelope, SOAP Header และ SOAP Body



รูปที่ 3-3 โครงสร้างของ SOAP

SOAP เมสเสจ จะต้องเป็นไปตามกฎนี้

- Envelope เป็นอีลีเมนต์ที่อยู่บนสุด ต้องมีอีลีเมนต์นี้เสมอ
- Header อาจจะมีหรือไม่มีก็ได้ แต่ถ้ามีต้องเป็น child element แรกของ Envelope
- Body ต้องเป็น child element แรกของ envelope หรือ header

ตัวอย่างของ SOAP เมสเสจ ตัวอย่างนี้คือ GetLastTradePrice SOAP Request ที่ส่งไปยัง StockQuote service ประกอบด้วยสตริงพารามิเตอร์ symbol และกิ้นค่า float กลับมากับ SOAP response SOAP envelope element อยู่ที่จุดบนสุดของเอกสาร XML XML นามสเปซ ใช้เพื่อป้องกันการสับสนของ SOAP identifier

SOAP request จะเป็นดังนี้

POST /StockQuote HTTP/1.1

Host: www.stockquoteserver.com

```

Content-Type: text/xml; charset="utf-8"
Content-Length: nnnn
SOAPAction: "Some-URI"

<SOAP-ENV:Envelope
  xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/"
  SOAP-ENV:encodingStyle="http://schemas.xmlsoap.org/soap/encoding/">
  <SOAP-ENV:Body>
    <m:GetLastTradePrice xmlns:m="Some-URI">
      <symbol>DIS</symbol>
    </m:GetLastTradePrice>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>

```

SOAP response จะเป็นดังนี้

```

HTTP/1.1 200 OK
Content-Type: text/xml; charset="utf-8"
Content-Length: nnnn

<SOAP-ENV:Envelope
  xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/"
  SOAP-ENV:encodingStyle="http://schemas.xmlsoap.org/soap/encoding/">
  <SOAP-ENV:Body>
    <m:GetLastTradePriceResponse xmlns:m="Some-URI">
      <Price>34.5</Price>
    </m:GetLastTradePriceResponse>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>

```

นอกจากนั้นใน SOAP เมสเสจ จะมีการใช้ XML เนมสเปซ ทุกๆ อีลีเมนต์ในเอกสารจะขึ้นต้นด้วย เนมสเปซ เนมสเปซ จะถูกกำหนดโดยใช้ xmlns attribute ดังนี้

```
<SOAP-ENV:Envelope
```

```
xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
```

แต่หากไม่ต้องการให้ขึ้นต้นด้วย เนมสเปซ ก็สามารถทำได้ ดังนี้

```
<Envelope
```

```
xmlns="http://schemas.xmlsoap.org/soap/envelope/">
```

3.2.2.1 Envelope

envelope element เป็นอีลีเมนต์บนสุดของเอกสาร XML ที่ใช้แสดงเมสเสจ อาจประกอบด้วยแอตทริบิวต์ คือ envelope เนมสเปซ และ encodingStyle

1. Envelope เนมสเปซ – SOAP เมสเสจจะแสดงเวอร์ชันโดยใช้ เนมสเปซ ของ envelope element เนมสเปซ จะถูกนำไปใช้ขึ้นต้น envelope, header และ body element
2. EncodingStyle Attribute – ใช้แสดงวิธี serialization ของ SOAP เมสเสจ แอตทริบิวต์นี้สามารถมีได้ในทุกอีลีเมนต์และจะมีผลกับ content ของอีลีเมนต์นั้นและ child element ทั้งหมดของมันที่ไม่ได้ประกาศแอตทริบิวต์

3.2.2.2 Body

เป็นส่วนของข้อมูลที่ใช้ในการแลกเปลี่ยน โดยทั่ว ๆ ไปข้อมูลจะเป็นการ marshall RPC call และ error report child element ของ body element จะถูกเรียกว่า Body entry

body entry จะต้องเป็นไปตามกฎนี้

- body entry จะต้องแสดงด้วยชื่อเต็มประกอบด้วย เนมสเปซ URI และ ชื่อของมัน (local name)
- SOAP encodingStyle attribute อาจจะมีได้ เพื่อแสดงถึงวิธี encode ของ body entry นั้น

3.2.2.3 Header

เป็นส่วนเพิ่มเติมของเมสเสจสามารถประกอบด้วยอินฟอร์เมชันที่ระบุไปยังแอปพลิเคชัน ส่วนเพิ่มเติมนี้อาจนำไปใช้implements เป็น authentication, transaction management เป็นต้น ทุก ๆ child element ของ header element จะถูกเรียกว่า Header entry

header entry จะต้องเป็นไปตามกฎดังนี้

- header entry จะต้องแสดงด้วยชื่อเต็มประกอบด้วย เนมสเปซ URI และ ชื่อของมัน (local name)
- อาจมี SOAP encodingStyle attribute ที่ใช้สำหรับ header entry
- SOAP mustUnderstand attribute และ SOAP actor attribute จะมีหรือไม่ก็ได้ เพื่อใช้บอกว่าจะจัดการกับ entry นั้นอย่างไรและโดยใคร

```

<soap:Header>

  <m:local xmlns:m="http://www.w3schools.com/local/"

    soap:actor="http://www.w3schools.com/appml" />

    <m:language>en</m:language>

    <m:currency>USD</m:currency>

  </m:local>

</soap:Header>

```

- 1) SOAP Actor Attribute – SOAP เมสเสจสามารถถูกส่งไปยังปลายทางผ่านตัวกลางของ SOAP (SOAP intermediary) ตัวกลางของ SOAP คือแอปพลิเคชันที่มีความสามารถทั้งรับและส่งต่อ SOAP เมสเสจ ในการส่งต่อตัวกลางจะต้องไม่ส่งต่อ header element ไปให้กับแอปพลิเคชัน โดยตัวกลางนี้จะถูกกำหนดเป็น URI ถ้าไม่มี attribute นี้หมายถึงผู้รับ SOAP เมสเสจเป็นปลายทางสุดท้าย
- 2) SOAP MustUnderstand Attribute – ใช้แสดงว่า header entry นี้จำเป็นหรือเป็นเพียงข้อป้อน สำหรับผู้รับที่จะนำไปประมวลผล ค่าของมันคือ “1” หรือ “0” ถ้าไม่มีการระบุจะมีค่าเท่ากับ “0” โดย “0” หมายถึงเป็นข้อป้อน และ “1” หมายถึงจำเป็น โดยจะต้องประมวลผลให้ถูกต้องตาม semantic หรือต้อง fail เช่น เมสเสจเป็นส่วนหนึ่งของ transaction ถ้าปลายทางไม่รองรับ transaction จะต้องไม่ประมวลผลและส่งผลผิดพลาดกลับไปในกรณีที่มี mustUnderstand attribute เป็น “1” แต่ถ้าเป็น “0” จะยังคงประมวลผลต่อไป

3.2.3 SOAP Fault Element

ข้อความแสดงความผิดพลาดจากแอปพลิเคชันของ SOAP จะเก็บอยู่ใน fault element ซึ่งถ้ามี จะต้องปรากฏใน body element เพียงครั้งเดียวใน SOAP เมสเสจ ตัวอย่างอาจเป็นดังนี้

```

<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"

  soap:encodingStyle="http://schemas.xmlsoap.org/soap/encoding/">

  <soap:Body>

    <soap:Fault>

      <faultcode>soap:MustUnderstand</faultcode>

      <faultstring>Mandatory Header error.</faultstring>

      <faultactor>http://www.wrox.com/heroes/endpoint.asp</faultactor>

      <detail>

        <w:source xmlns:w="http://www.wrox.com/">

```

```

    <module>endpoint.asp</module>

    <line>203</line>

  </w:source>

</detail>

</soap:Fault>

</soap:Body>

</soap:Envelope>

```

SOAP fault element มี element ย่อย ๆ ดังตารางที่ 3-5

Sub Element	Description
<faultcode>	โค้ดที่ระบุถึงการ error
<faultstring>	ข้อความการ error
<faultactor>	ใครเป็นสาเหตุของการ error
<detail>	ระบุนิพจน์ของ error

ตารางที่ 3-5 SOAP Fault Element

ค่าของ fault code สามารถมีค่าได้ดังตารางที่ 3-6

Error	Description
VersionMismatch	เนมสเปซ ภายใน SOAP Envelope element ไม่ถูกต้อง
MustUnderstand	child element ของ Header element กับ mustUnderstand attribute ที่มีค่า "1" ผู้รับไม่รองรับ
Client	เมสเสจมีรูปแบบไม่ถูกต้อง หรือมีอินฟอร์เมชันที่ไม่ถูกต้อง
Server	เกิดปัญหากับเซิร์ฟเวอร์ ไม่สามารถประมวลผลได้

ตารางที่ 3-6 ค่า fault code

3.2.4 SOAP Encoding

SOAP encoding มีวิธีการ map จาก type ของโปรแกรมมิ่งไปเป็น XML 2 วิธี คือจากภายนอก โดยใช้ WSDL (Web Services Description Language) ที่บอกถึง type ของข้อมูลที่รับหรือส่ง หรือใช้ xsi:type attribute ในกรณีที่ภาษาที่ใช้ไม่รองรับ WSDL โดยทั้ง 2 วิธีจะใช้ XML schema ในการระบุ type ตัวอย่างของการใช้ xsi จะเป็นดังนี้

```
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
  soap:encodingStyle="http://schemas.xmlsoap.org/soap/encoding/"
  xmlns:xsi="http://www.w3.org/1999/XMLSchema-instance"
  xmlns:xsd="http://www.w3.org/1999/XMLSchema">
  <soap:Body>
    <m:MixedMessage xmlns:m="http://www.wrox.com/mix/">
      <param1 xsi:type="xsd:string">OU812</param1>
      <param2 xsi:type="xsd:integer">2001</param2>
      <param3 xsi:type="xsd:double">3.14159</param3>
    </m:MixedMessage>
  </soap:Body>
</soap:Envelope>
```

SOAP รองรับ type ของข้อมูลได้ทั้ง simple type และ compound type เช่น struct และ array

3.2.4.1 ตัวอย่าง Struct

struct book ที่เขียนด้วยภาษา c++ จะเป็นดังนี้

```
struct Book
{
  string author;
  string name;
  int page;
};
```

Soap เมสเซจที่ไม่ใช่ WSDL แต่ใช้ xsi จะเป็นดังนี้

```
<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/"
  xmlns:xsi="http://www.w3.org/1999/XMLSchema-instance">
```

```

xmlns:xsd="http://www.w3.org/1999/XMLSchema">
<SOAP-ENV:Body>
<ns1:searchBook xmlns:ns1="http://soapinterop.org/" SOAP-
ENV:encodingStyle="http://schemas.xmlsoap.org/soap/encoding/">
<bookResult xmlns:ns2="http://soapinterop.org/xsd" xsi:type="ns2:Book">
  <author xsi:type="xsd:string">Ed Roman</author>
  <name xsi:type="xsd:string">Mastering EJB</intro>
  <page xsi:type="xsd:int">500</page>
</bookResult>
</ns1:searchBook>
</SOAP-ENV:Body>
</SOAP-ENV:Envelope>

```

แต่ถ้าใช้ WSDL ในส่วนของการประกาศ schema จะต้องประกอบด้วย schema ดังนี้

```

<types>
  <schema targetNamespace="http://soapinterop.org/xsd"
    xmlns="http://www.w3.org/2001/XMLSchema"
    xmlns:SOAP-ENC="http://schemas.xmlsoap.org/soap/encoding/"
    xmlns:wSDL="http://schemas.xmlsoap.org/wSDL/"
    elementFormDefault="qualified">
    <complexType name="Book">
      <sequence>
        <element name="author" type="string"/>
        <element name="name" type="string"/>
        <element name="page" type="int"/>
      </sequence>
    </complexType>
  </schema>
</types>

```

3.2.4.2 ตัวอย่าง Array

ตัวอย่างนี้เป็น array ของ struct book ในตัวอย่างก่อน Soap เมสเสจที่ไม่ใช่ WSDL แต่ใช้ xsi จะเป็นดังนี้

```
<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/"
  xmlns:xsi="http://www.w3.org/1999/XMLSchema-instance"
  xmlns:xsd="http://www.w3.org/1999/XMLSchema">
  <SOAP-ENV:Body>
    <ns1:BookArrayResult xmlns:ns1="http://soapinterop.org/" SOAP-
      ENV:encodingStyle="http://schemas.xmlsoap.org/soap/encoding/">
      <BookArray xmlns:ns2="http://schemas.xmlsoap.org/soap/encoding/"
        xsi:type="ns2:Array" xmlns:ns3="http://soapinterop.org/xsd"
        ns2:arrayType="ns3:Book[3]">
        <item xsi:type="ns3:Book">
          <author xsi:type="xsd:string">Ed Roman</author>
          <name xsi:type="xsd:string">Mastering EJB</name>
          <page xsi:type="xsd:int">500</page>
        </item>
        <item xsi:type="ns3:Book">
          <author xsi:type="xsd:string">Roger Session</author>
          <name xsi:type="xsd:string">The Battle of the middleware</name>
          <page xsi:type="xsd:int">350</page>
        </item>
        <item xsi:type="ns3:Book">
          <author xsi:type="xsd:string">Chris Dix</author>
          <name xsi:type="xsd:string">Professional XML Web Service</name>
          <page xsi:type="xsd:int">550</page>
        </item>
      </BookArray>
    </ns1:BookArrayResult>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

แต่ถ้าใช้ WSDL ในส่วนของการประกาศ schema จะต้องประกอบด้วย schema ดังนี้

```
<types>

  <schema targetNamespace='http://soapinterop.org/xsd'
    xmlns='http://www.w3.org/2001/XMLSchema'
    xmlns:SOAP-ENC='http://schemas.xmlsoap.org/soap/encoding/'
    xmlns:wSDL='http://schemas.xmlsoap.org/wSDL/'
    elementFormDefault='qualified'>

    <complexType name="Book">
      <sequence>
        <element name='author' type='string'/>
        <element name='name' type='string'/>
        <element name='page' type='int'/>
      </sequence>
    </complexType>

    <complexType name='ArrayOfBook'>
      <complexContent>
        <restriction base='SOAP-ENC:Array'>
          <attribute ref='SOAP-ENC:arrayType' wsdl:arrayType='typens:Book[]'/>
        </restriction>
      </complexContent>
    </complexType>
  </schema>
</types>
```

3.2.5 SOAP ใน HTTP

ในการส่ง SOAP ผ่านทาง HTTP จะต้องใช้ content-type เป็น text/xml แต่ใน SOAP request จะต้องใช้ header SOAPAction ภายใน HTTP header SOAPAction จะเป็นตัวบอกให้เซิร์ฟเวอร์รู้ว่า HTTP Post นั้นเป็น SOAP เมสเสจ และค่าของ header คือ URI ที่แสดงถึงจุดหมายของ SOAP เมสเสจ ส่วน SOAP response จะต้องใช้ status code ตามมาตรฐานของ HTTP โดย 200-299 แสดงว่าสำเร็จ แต่ถ้า response เมสเสจ เป็นการ fault แล้ว status code จะต้องเป็น 500 ซึ่งแสดงถึง internal server error ตัวอย่างของ response ที่มี status code เป็น 500 อาจเป็นดังนี้

```

HTTP/1.1 500 Internal Server Error
Content-Type: text/xml
Content-Length: ###
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
    soap:encodingStyle="http://schemas.xmlsoap.org/soap/encoding/"
    xmlns:xsi="http://www.w3.org/1999/XMLSchema-instance"
    xmlns:xsd="http://www.w3.org/1999/XMLSchema">
  <soap:Body>
    <soap:Fault>
      <faultcode>soap:VersionMismatch</faultcode>
      <faultstring>The SOAP เนมสเปซ is incorrect.</faultstring>
      <faultactor>http://www.wrox.com/endpoint.asp</faultactor>
      <detail>
        <w:errorinfo xmlns:w="http://www.wrox.com/">
          <desc>The SOAP เนมสเปซ was blank.</desc>
        </w:errorinfo>
      </detail>
    </soap:Fault>
  </soap:Body>
</soap:Envelope>

```

3.2.6 SOAP สำหรับ RPC

จุดประสงค์ของการออกแบบ SOAP คือทำ RPC โดยใช้ XML การเรียก RPC นั้นจะ map เข้ากับ

HTTP request และ RPC response จะ map กับ HTTP response

ในการทำ method call จำเป็นที่จะต้องมียีนฟอร์เมชันดังนี้

1. URI ของออบเจกต์ปลายทาง (target object)
2. ชื่อของเมธอด
3. คำอธิบายของเมธอด (method signature) ส่วนนี้เป็นอ็อปชัน
4. พารามิเตอร์สำหรับเมธอด
5. ข้อมูลส่วนหัว (header data) เป็นอ็อปชัน

พิจารณาเมธอดดังนี้

```
double GetStockQuote ( [in] stirng sSymbol);
```

ถ้า เนมสเปซ ของเมธอดคือ “http://www.worxstox.com/” แล้วการเรียกเมธอด โดย request stock quote ใช้ symbol OU812 จะเป็นดังนี้

```
<q:GetStockQuote xmlns:q="http://www.wroxstox.com/">
  <q:sSymbol xsi:type="xsd:string">OU812</q:sSymbol>
</q:GetStockQuote>
```

ชื่อเมธอดและชื่อของอีลีเมนต์จะต้อง match กันเช่นเดียวกับพารามิเตอร์

สำหรับ response จะต้องชื่ออีลีเมนต์จะต้องเป็นชื่อเมธอดตามด้วย response ดังนี้

```
<q:GetStockQuoteResponse xmlns:q="http://www.wroxstox.com/">
  <q:ret xsi:type="xsd:double">100.0</q:ret>
</q:GetStockQuoteResponse>
```

ชื่ออีลีเมนต์ของค่าที่คืนมา สามารถเปลี่ยนได้โดยจะกำหนดได้จาก โปรแกรมที่เขียนหรือจาก WSDL

3.2.7 SOAP Toolkit

SOAP toolkit คือเครื่องมือที่จะทำหน้าที่ในการประมวลผล SOAP request และส่ง response กลับไปให้ไคลเอนต์ โดย SOAP toolkit แต่ละตัวใช้ได้บนแพลตฟอร์มที่ต่างกัน รองรับเทคโนโลยี เว็บเซอร์วิส ต่างกัน บางตัวอาจจะรองรับ WSDL หรือ UDDI ในขณะที่บางตัวรองรับอย่างใดอย่างหนึ่ง หรือไม่รองรับทั้งสองอย่าง และถึงแม้ว่า SOAP จะเป็นอิสระต่อแพลตฟอร์ม แต่ใน SOAP toolkit บางตัว ยังไม่สามารถทำงานข้ามผลิตภัณฑ์ (interoperability) ได้ เนื่องจากรองรับเทคโนโลยีของ เว็บเซอร์วิส ไม่เหมือนกัน เช่น xsi ในบางผลิตภัณฑ์จะบังคับให้ SOAP เมสเสจจะต้องระบุ type ด้วย xsi หากไม่มีก็จะ failut ซึ่งในปัจจุบันแต่ละผลิตภัณฑ์ก็พยายามพัฒนาให้สามารถทำงานด้วยกันได้

ชื่อ	แพลตฟอร์ม	แหล่งที่มา
4s4c	COM	http://www.4s4c.com
A SOAP for RPC NT Service	COM	http://www.whitemesa.com
Apache Axis	Java	http://xml.apache.org/axis
Apache SOAP	Java	http://xml.apache.org/soap
CapeConnect	Java	http://www.capeclear.com
Glue	Java	http://www.themindelectric.com
IBM Web Services Toolkit	Java	http://www.alphaworks.ibm.com
IdooXoap for Java and C++	Java , C++	http://www.zvon.org
Microsoft SOAP Toolkit	COM	http://www.microsoft.com
PocketSOAP	COM	http://www.pocketsoap.com
SOAP::Lite	Perl	http://www.soaplite.com
Visual Studio .NET	COM	http://www.microsoft.com

ตารางที่ 3-7 SOAP Toolkit

3.2.8 โครงสร้างภายในของ SOAP Toolkit ของ Microsoft

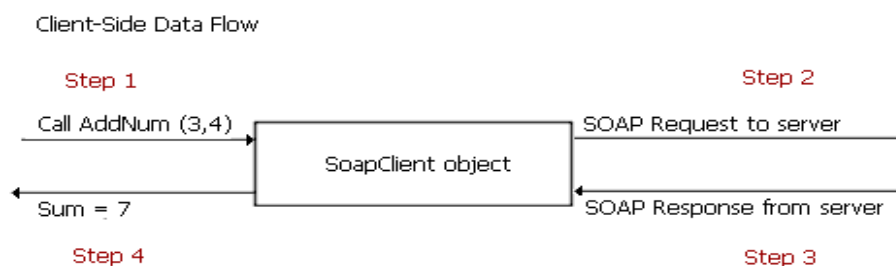
The Web Services Description Language (WSDL) เป็นรูปแบบหนึ่งของ XML เพื่อใช้อธิบายว่าเซิร์ฟเวอร์นั้นมีเซอร์วิส (Services) และ operation ภายในเซอร์วิสอะไรให้ใช้บ้าง ซึ่ง operation ภายในเซอร์วิสนั้นจะอธิบายรูปแบบที่ไคลเอนต์ (client) ต้องใช้ในการเรียก operation

The Web Services Meta Language (WSML) file เป็นส่วนที่เพิ่มขึ้นมาจากไฟล์ WSDL เพื่อใช้ในการเชื่อมโยงแต่ละ operation ในเซอร์วิสกับ COM object โดย WSML file จะพิจารณาว่า COM object ตัวใดที่จะต้องถูกโหลด (load) ขึ้นมาเมื่อมีการเรียกใช้ operation ในเซอร์วิส

ทั้งไฟล์ WSDL และ WSML จะต้องอยู่ที่เซิร์ฟเวอร์ โดยไคลเอนต์ที่ต้องการที่จะส่ง SOAP request ไปที่เซิร์ฟเวอร์ จะต้องได้รับ WSDL จากเซิร์ฟเวอร์ก่อน เพื่อหาารูรูปแบบของ SOAP request ในการติดต่อ

3.2.8.1 การสร้าง SoapClient Object

ทางฝั่งไคลเอนต์ เมื่อแอปพลิเคชันส่ง message ไปที่ Soap Client object เพื่อเรียกใช้ operation หนึ่งๆ นั้น(ตามรูปนี้คือการบวกเลข 2 ตัว) SoapClient object จะทำการสร้าง SOAP request ส่งไปที่เซิร์ฟเวอร์ เมื่อเซิร์ฟเวอร์ทำการประมวลผลเสร็จเรียบร้อยแล้ว ก็จะส่งผลลัพธ์ในรูปแบบของ SOAP response กลับมาให้ไคลเอนต์ หลังจากนั้น SoapClient object จึงจะทำการแปลและส่ง message ที่เก็บผลลัพธ์ของ operation นั้นกลับไปให้แอปพลิเคชัน ตามไดอะแกรมด้านล่าง



รูปที่ 3-4 การ Client-Side Data Flow

แอปพลิเคชันต้อง instantiate Soap object ก่อนที่จะทำการเรียกใช้ operation ใดๆ (SoapClient object นี้ถูกสร้างเก็บไว้ในไฟล์ MSSOAP1.dll โดยใช้เมธอด SoapClient.mssoapinit (ชื่อไฟล์ WSDL, ชื่อเซอร์วิส, ข้อมูลพอร์ต) โดยข้อมูลพอร์ตจะใช้ในการแบ่งแยกช่องทางที่ใช้ในการส่ง SOAP message

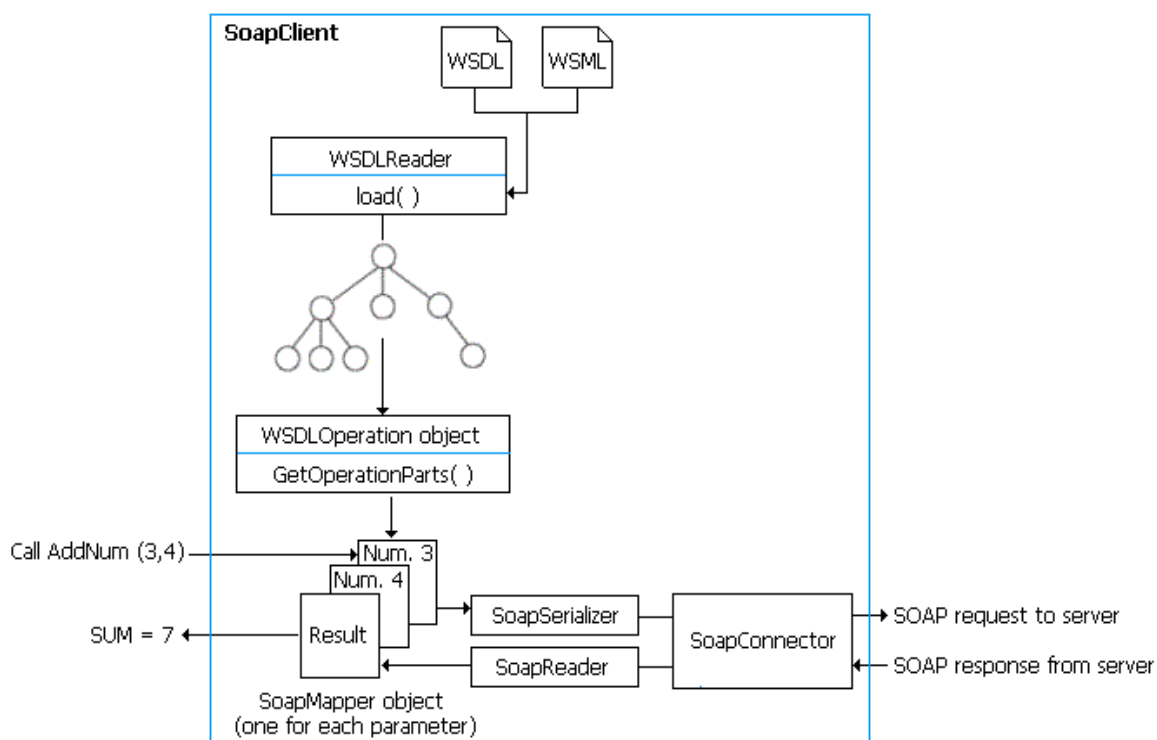
ในระหว่าง initialize SoapClient object จะตรวจสอบเมธอดทั้งหมดที่อยู่ในไฟล์ WSDL ด้วยการทำงานแบบไดนามิกนี้ทำให้เราสามารถที่จะเรียกใช้เมธอดใดก็ได้ที่ได้อธิบายไว้ในไฟล์ WSDL ดังแสดงตาม Microsoft Visual Basic Scripting Edition (VBScript) code ด้านล่างนี้

```

SET soapclient = CreateObject("MSSOAP.SoapClient")
Call soapclient.mssoapinit("http://localhost/MyWSDL.wsdl", "TestService", "TestPort")
' Now the client can call an operation listed in the portType element
' specified when calling mssoapinit().
wscript.echo soapclient.AddNumbers(2, 3)
  
```


3.2.8.2 การประมวลผลภายใน SoapClient Object

การทำงานภายใน SoapClient object แสดงได้ดังรูปที่ 3-5



รูปที่ 3-5 โครงสร้างภายในของ SOAP Client

WSDL Reader object จะทำการโหลดไฟล์ WSDL และไฟล์ WSML เข้าไปใน DOM แล้วทำการวิเคราะห์ โดย WSDLReader object จะสร้าง WSDLOperation object เพื่อทำการเรียกใช้งาน operation (ตามรูปนี้คือ AddNum(3, 4)) หลังจากนั้น WSDLOperation จะเรียกเมธอด GetOperationParts เพื่อดึงรูปแบบ message ของอินพุตและเอาที่พุทของ operation ออกมา แล้ว SoapClient จะสร้าง Soap Mapper objects ให้กับแต่ละส่วนที่ได้จากเมธอด GetOperationParts และโหลดค่าที่ได้มาจากการเรียกใช้ operation เข้าไว้ใน objects เหล่านี้ พอถึงขั้นตอนนี้ SoapSerializer objects จะสร้าง SOAP request message จาก SoapMapper objects ที่เหมาะสมและส่งไปที่เซิร์ฟเวอร์

เมื่อเซิร์ฟเวอร์ประมวลผลเสร็จแล้วก็จะส่ง SOAP response กลับไปที่ไคลเอนต์ SoapClient object จะทำการโหลดผลลัพธ์ของ operation ไปเก็บใน SoapMapper object ที่เหมาะสมแล้วส่งผลลัพธ์กลับไปให้แอปพลิเคชันของผู้ใช้

3.2.8.3 การสร้าง SoapServer Object

ส่วนทางฝั่งเซิร์ฟเวอร์ เราสามารถกำหนดรูปแบบการรับ SOAP request (SOAP listener) ได้ 2 รูปแบบ คือ Internet Server API (ISAPI) server และ Active Server Pages (ASP) server

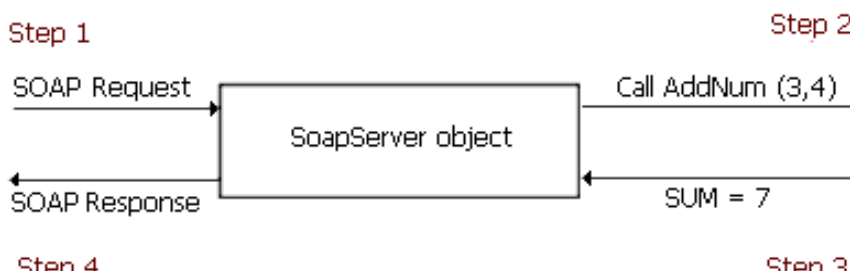
ในไฟล์ WSDL จะมี URL ที่ระบุว่าจะใช้ไฟล์ใดบนเซิร์ฟเวอร์ในการรับ SOAP request เพื่อใช้ในการพิจารณาว่าจะรับ SOAP request อย่างไรบนเซิร์ฟเวอร์ ตามโค้ดด้านล่างนี้ ตัวพิมพ์หนาจะเป็นส่วนที่ใช้ในการเรียกไปที่ ISAPI listener

```
<definitions>
...
<service name='DocSample1' >
  <port name='DocSample1PortType' binding='tns:DocSample1Binding' >
    <soap:address
      location='http://localhost/DocSample1Test/DocSample1.wsdl' />
  </port>
</service>
...
</definitions>
```

การทำงานทางฝั่งเซิร์ฟเวอร์

ไม่ว่า SOAP request จะเรียกไปยัง ISAPI หรือ ASP listener ก็ตาม ข้อมูลที่รับส่งก็ยังคงเหมือนกัน เมื่อ SoapServer object ได้รับ SOAP request จากไคลเอนต์ ก็จะทำการแปลและเรียกไปยังเมธอด COM ที่ตรงกับ operation ที่ต้องการ (ตามไดอะแกรมนี้คือเมธอด AddNum) หลังจากนั้นเมธอดที่ถูกเรียกใช้งานก็จะส่งผลลัพธ์กลับมาให้ SoapServer object ซึ่งมันก็จะแปลงผลลัพธ์ให้อยู่ในรูปแบบของ SOAP response แล้วจึงส่งกลับไปที่ไคลเอนต์

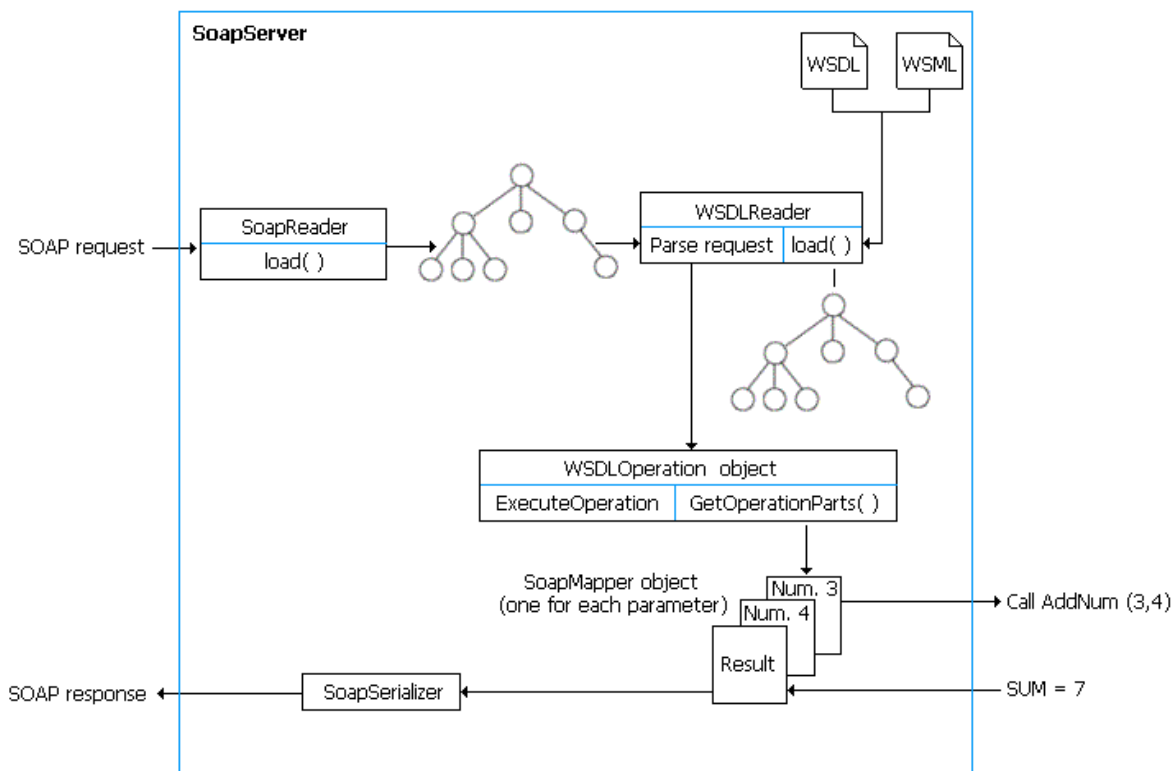
Server-Side Data Flow



รูปที่ 3-6 Server-Side Data Flow

3.2.8.4 การประมวลผลภายใน SoapServer Object

การทำงานภายใน SoapServer object แสดงได้ดังรูปด้านล่าง



รูปที่ 3-7 โครงสร้างภายใน SOAP Server

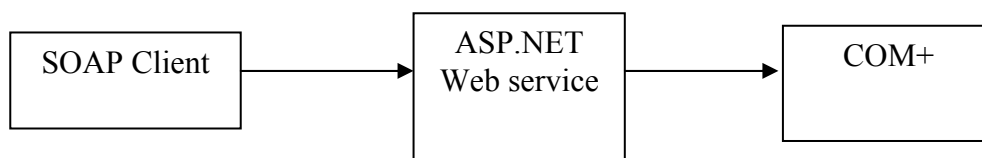
เมื่อได้รับ SOAP request จากไคลเอนต์แล้ว SoapReader object ก็จะโหลด request message ไปไว้ใน DOM structure หนึ่ง ในขณะที่ WSDLReader จะโหลดไฟล์ WSDL และไฟล์ WSML ไปไว้ในอีก DOM structure หนึ่ง ตัว WSDLReader จะแปล request และสร้าง WSDLOperation object ขึ้นเพื่อใช้ใน requested operation หลังจากนั้น WSDLOperation object จะเรียกใช้เมธอด GetOperationParts เพื่อดึงรูปแบบ message ของอินพุตและเอาต์พุตของ operation ออกมา แล้ว SoapServer จะสร้าง Soap Mapper objects ให้กับแต่ละส่วนที่ได้จากเมธอด GetOperationParts และโหลดค่าที่ได้มาจากการเรียกใช้ operation เข้าไว้ใน objects เหล่านี้ พอถึงขั้นตอนนี้ SoapServer object จะทำการเรียกไปที่ COM method ที่ตรงกับที่ร้องขอเข้ามา

เมื่อ COM method ประมวลผลเสร็จแล้วก็จะส่งผลลัพธ์กลับไป SoapServer object ซึ่ง SoapServer object ก็จะทำการโหลดผลลัพธ์ของ operation ไปเก็บใน SoapMapper object ที่เหมาะสม แล้วจึงใช้ SoapSerializer object ส่ง SOAP response message กลับไปให้ไคลเอนต์แอปพลิเคชันของผู้ใช้

3.2.9 การสร้าง SOAP Client และ SOAP Server ด้วย .NET Framework

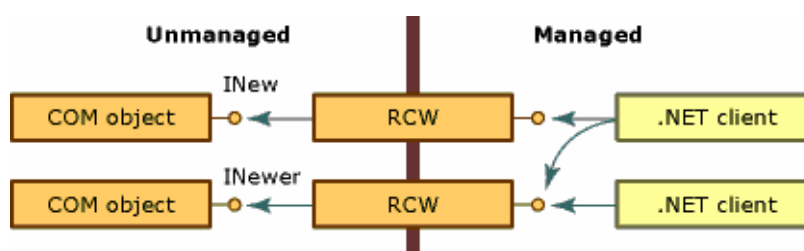
3.2.9.1 การใช้ ASP.NET ทำ SOAP เว็บเซอร์วิส

ใช้ความสามารถในการทำ COM+ Interop โดยสร้างคลาสของ ASP.NET มาเรียกใช้งาน business logic ที่สร้างด้วย Visual Basic 6



รูปที่ 3-8 การใช้ ASP.NET ทำ SOAP เว็บเซอร์วิส

อาศัยความสามารถในการทำ marshal / unmarshal object ให้อยู่ในรูปแบบ SOAP ของ ASP.NET โดย ASP.NET ต้องทำ Interop โดยอาศัย Runtime Callable Wrapper ช่วยจำลอง COM object ให้ .NET มองเห็นเป็น .NET object ธรรมดา



รูปที่ 3-9 Runtime Callable Wrapper

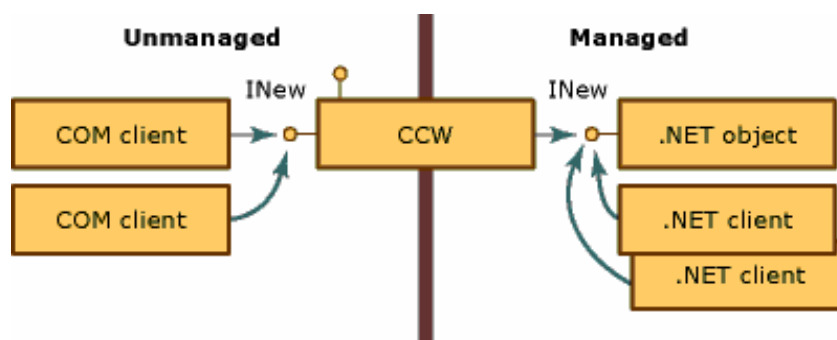
3.2.9.2 การใช้ .NET ทำ SOAP Client

ในทางกลับกันที่ COM ต้องการใช้บริการจาก SOAP ก็กระทำผ่าน .NET เว็บเซอร์วิส client



รูปที่ 3-10 การใช้ .NET ทำ SOAP client

แต่การที่จะทำให้ COM ติดต่อกับ .NET class ได้นั้นต้องผ่านกระบวนการ COM Interop โดยการสร้าง COM callable wrapper ซึ่งจะช่วยให้ COM สามารถเห็น .NET class เป็น COM object ธรรมดา



รูปที่ 3-11 COM callable wrapper ที่ทำให้ COM client สามารถเห็น .NET object

3.3 UDDI

UDDI (Universal Description, Discovery and Integration) เป็นมาตรฐานที่จัดตั้งขึ้น โดยบริษัท IBM, Microsoft และบริษัทยักษ์ใหญ่ทางธุรกิจ B2B (Business-to-Business) อื่น ๆ UDDI ถูกสร้างขึ้นมาเป็นมาตรฐาน ในการค้นหาบริการ เว็บเซอร์วิส สำหรับคู่ค้าทางธุรกิจ ซึ่งเปรียบได้กับฐานข้อมูล ขนาดใหญ่ ซึ่งมีข้อมูลของ เว็บเซอร์วิส ที่เปิดให้บริการ โดยที่ web site สำหรับค้นหาเว็บเซอร์วิส มีอยู่หลายที่ อย่างเช่น

- <http://uddi.microsoft.com/search.aspx>
- <http://www-3.ibm.com/services/uddi/testregistry/find>
- <http://uddi.org>

3.4 WSDL

WSDL (Web Services Description Language) เกิดจากความร่วมมือระหว่าง IBM และ Microsoft WSDL เป็นภาษาที่ใช้อธิบายคุณลักษณะของ เว็บเซอร์วิส และวิธีการติดต่อกับ เว็บเซอร์วิส นั้น ๆ โดยใช้ไวยากรณ์ของภาษา XML ซึ่ง WSDL อยู่ในความดูแลของ W3C version ล่าสุด คือ WSDL 1.1 รายละเอียด สามารถหาอ่านเพิ่มเติมได้จากเว็บไซต์ของ W3C ที่ <http://www.w3.org/TR/wsdl> ส่วนในทางปฏิบัติ หากเราต้องการ สร้าง เว็บเซอร์วิส ขึ้นมาเป็นของตนเอง ก็สามารถสร้างเอกสาร WSDL ได้ โดยอัตโนมัติ เราจึงไม่ต้องไปกังวลในรายละเอียด ในข้อกำหนดใน WSDL มากนักโดยตัวอย่างเอกสาร WSDL เช่น

```
<?xml version="1.0"?>
<definitions name="StockQuote" target namespace="http://example.com/stockquote.wsdl"
  xmlns:tns="http://example.com/stockquote.wsdl"
  xmlns:xsd1="http://example.com/stockquote.xsd"
  xmlns:soap="http://schemas.xmlsoap.org/wsdl/soap/"
```

```

xmlns="http://schemas.xmlsoap.org/wsdl/">
<types>
  <schema target namespace="http://example.com/stockquote.xsd"
    xmlns="http://www.w3.org/2000/10/XMLSchema">
    <element name="TradePriceRequest">
      <complexType>
        <all>
          <element name="tickerSymbol" type="string"/>
        </all>
      </complexType>
    </element>
    <element name="TradePrice">
      <complexType>
        <all>
          <element name="price" type="float"/>
        </all>
      </complexType>
    </element>
  </schema>
</types>
<message name="GetLastTradePriceInput">
  <part name="body" element="xsd1:TradePriceRequest"/>
</message>
<message name="GetLastTradePriceOutput">
  <part name="body" element="xsd1:TradePrice"/>
</message>
<portType name="StockQuotePortType">
  <operation name="GetLastTradePrice">
    <input message="tns:GetLastTradePriceInput"/>
    <output message="tns:GetLastTradePriceOutput"/>
  </operation>
</portType>
<binding name="StockQuoteSoapBinding" type="tns:StockQuotePortType">
  <soap:binding style="document" transport="http://schemas.xmlsoap.org/soap/http"/>

```

```
<operation name="GetLastTradePrice">
  <soap:operation soapAction="http://example.com/GetLastTradePrice"/>
  <input>
    <soap:body use="literal"/>
  </input>
  <output>
    <soap:body use="literal"/>
  </output>
</operation>
</binding>
<service name="StockQuoteService">
  <documentation>My first service</documentation>
  <port name="StockQuotePort" binding="tns:StockQuoteBinding">
    <soap:address location="http://example.com/stockquote"/>
  </port>
</service>
</definitions>
```